UNIVERSIDAD DE CARTAGENA

Ingeniería de Software · Periodo 2026-2

Sistema de nómina orientado a objetos

Informe técnico y evidencias de ejecución

Asignatura	Ingeniería de Software B1
Unidad	Unidad 3
Docente	Carlos Jose Caceres Ochoa (según tarjeta del curso en SIMA)
Estudiante	Nicolás Andrés Arnedo Puello
Modalidad	Desarrollo individual
Fecha de preparación	31 de octubre de 2026

Realicé el análisis, el diseño, la implementación, las pruebas y la documentación de forma individual.

Resumen

Realicé de forma individual esta actividad de la Unidad 3 y desarrollé un sistema de nómina en Java. El modelo implementa cuatro tipos de empleado, aplica las reglas expresas, valida entradas y evita salarios netos negativos. En la ejecución local del 31 de octubre de 2026, las 10 pruebas unitarias finalizaron sin fallas ni errores.

El código, las pruebas, el informe y la exposición corresponden al trabajo que realicé individualmente. Para los valores que no estaban definidos en el enunciado adopté supuestos de demostración.

1. Alcance y trazabilidad

Requisito	Implementación o evidencia	Resultado
Programación orientada a objetos para cuatro tipos de empleado	Clases SalariedEmployee, HourlyEmployee, CommissionEmployee y TemporaryEmployee bajo el contrato Employee.	Implementado
Principios SOLID, código limpio y refactorización	Tipos cohesivos, interfaces estrechas y política de deducción inyectada.	Implementado
Comentarios de código	Comentarios breves en decisiones no obvias y reglas documentadas en metodología.	Incluido
Pruebas unitarias	JUnit Jupiter y Maven Surefire; 10 casos ejecutados.	10 aprobadas
Metodología de desarrollo	Secuencia de análisis, modelado, implementación y verificación.	Documentada
Repositorio del proyecto	Código fuente e historial de versiones en el repositorio público enlazado al final.	Disponible
Exposición en video	Enlace de YouTube incluido al final del informe.	Enlace disponible

2. Reglas de negocio implementadas

Tipos de empleado

- Asalariado: salario fijo mensual; suma un bono de 10% del salario fijo solo con más de cinco años de servicio. En cinco años exactos no aplica el bono.
- Por horas: paga hasta 40 horas a tarifa base; las horas posteriores se liquidan a 1.5 veces la tarifa normal. No recibe bonos.
- Por comisión: salario base más la comisión calculada con una tasa configurable. Si las ventas superan COP 20.000.000, agrega un bono de 3% de las ventas.
- Temporal: salario fijo y duración contractual positiva; no recibe beneficios ni bonos adicionales.

Deducciones y beneficios

- Seguro social y pensión: 4% combinado del salario bruto, según la consigna.
- ARL: se calcula con una tasa configurable sobre el salario bruto en el modelo.
- Alimentación: COP 1.000.000 para asalariados y empleados por comisión, separado del pago neto porque lo cubre la empresa.
- Ahorro: 2% para empleados por horas con más de un año y aceptación del fondo; se registra como aporte separado del efectivo neto.

3. Diseño y decisiones técnicas

El sistema separa las reglas por tipo de empleado. PayrollService compone el resultado y recibe DeductionPolicy mediante su constructor. PercentageDeductionPolicy aplica el 4% indicado y una tasa ARL provista desde configuración. El fondo de ahorro usa una interfaz aparte porque no aplica a todos los empleados.

```
Employee → { SalariedEmployee, HourlyEmployee, CommissionEmployee, TemporaryEmployee }
HourlyEmployee → SavingsFundParticipant
PayrollService → DeductionPolicy → PercentageDeductionPolicy
PayrollService.calculate(Employee) → PayrollResult
```

Aplicación de SOLID

Principio	Aplicación concreta
Responsabilidad única	Cada clase de empleado calcula sus ingresos; la política calcula deducciones; el servicio arma la liquidación; la demostración presenta los datos.
Abierto/cerrado	Un nuevo empleado puede implementarse como otro tipo Employee sin editar las reglas existentes.
Sustitución de Liskov	Los cuatro tipos entregan salario bruto y beneficios mediante el mismo contrato.
Segregación de interfaces	SavingsFundParticipant se implementa solo en el tipo que puede aportar al fondo.
Inversión de dependencias	PayrollService depende de DeductionPolicy y recibe la implementación por constructor.

Buenas prácticas y refactorización

Separé las reglas de negocio en tipos cohesivos; Money reúne validaciones y redondeo; DeductionPolicy desacopla la configuración de tasas; BigDecimal evita errores de precisión típicos de punto flotante para dinero. Las validaciones rechazan horas o ventas negativas y el servicio rechaza un neto inferior a cero.

4. Metodología aplicada

1. Análisis: extraje del enunciado actores, reglas y condiciones límite.
2. Diseño: definí el contrato Employee, tipos especializados y una política configurable de deducciones.
3. Implementación: codifiqué cada regla en el dominio y la composición del pago en PayrollService.
4. Verificación: añadí pruebas para los umbrales, errores de entrada y cálculos monetarios.
5. Demostración: ejecuté escenarios de los cuatro tipos con valores asumidos para los parámetros no definidos.

Realicé estas etapas individualmente, desde el análisis del enunciado hasta la ejecución de las pruebas y la documentación de resultados.

5. Pruebas unitarias

Caso	Comprobación
Antigüedad asalariado	10% solo con más de cinco años; límite de cinco años exactos.
Empleado por horas	40 horas regulares y horas extra a 1.5; 4% social/pensión; ARL configurable; ahorro opcional.
Elegibilidad de ahorro	Más de un año y consentimiento explícito.
Bono por comisión	3% solo al superar COP 20.000.000; el valor exacto del umbral no activa el bono.
Alimentación	Beneficio de empresa separado del salario neto.
Temporal	Salario fijo sin beneficios adicionales.
Validaciones	Horas y ventas negativas, tasas fuera de rango y neto negativo se rechazan.
Redondeo	Montos redondeados a dos decimales con HALF_UP.

La ejecución del 31 de octubre de 2026 reportó 10 pruebas ejecutadas, 0 fallas, 0 errores y 0 omitidas. El registro está en docs/evidence/pruebas_maven_real.txt.

6. Ejemplo de salida

La demostración utiliza valores asumidos para los parámetros no especificados en el enunciado. Los registros y nombres que aparecen son ficticios y se usan con fines académicos.

Tipo	Bruto	Beneficio empresa	SS + pensión	ARL demo	Ahorro	Neto
Asalariada	COP 4.400.000	COP 1.000.000	COP 176.000	COP 44.000	COP 0	COP 4.180.000
Por horas	COP 950.000	COP 0	COP 38.000	COP 9.500	COP 19.000	COP 883.500
Por comisión	COP 5.000.000	COP 1.000.000	COP 200.000	COP 50.000	COP 0	COP 4.750.000

Temporal	COP 1.800.000	COP 0	COP 72.000	COP 18.000	COP 0	COP 1.710.000

Para los parámetros no definidos en el enunciado adopté supuestos de demostración.

7. Capturas de evidencia

Estas capturas registran la salida original producida por las ejecuciones locales del sistema y de las pruebas el 31 de octubre de 2026. Las rutas absolutas de esta computadora se omitieron del registro publicado.

```
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------< co.edu.unicartagena:sistema-nomina-u3 >----------------
[INFO] Building Sistema de Nomina Unidad 3 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ sistema-nomina-u3 ---
[INFO] Deleting [ruta local del proyecto]\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ sistema-nomina-u3 ---
[INFO] skip non existing resourceDirectory [ruta local del proyecto]\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ sistema-nomina-u3 ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 14 source files with javac [debug release 17] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ sistema-nomina-u3 ---
[INFO] skip non existing resourceDirectory [ruta local del proyecto]\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ sistema-nomina-u3 ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.2:test (default-test) @ sistema-nomina-u3 ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running co.edu.unicartagena.nomina.PayrollServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.038 s -- in
co.edu.unicartagena.nomina.PayrollServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.847 s
[INFO] Finished at: 2026-10-31T15:50:15-05:00
[INFO] ------------------------------------------------------------------------
```

Figura 1. Salida de Maven: 10 pruebas ejecutadas y BUILD SUCCESS.

```
DEMO ACADEMICA - tasas no suministradas por la consigna pueden ser ilustrativas
Registros y nombres ficticios; no representan empleados reales.
ARL configurada: 1.0% | Comision configurada: 5.0%
Caso asalariado | bruto COP $ 4.400.000,00 | beneficio empresa COP $ 1.000.000,00 | SS+P COP $
176.000,00 | ARL COP $ 44.000,00 | ahorro COP $ 0,00 | neto COP $ 4.180.000,00
Caso por horas | bruto COP $ 950.000,00 | beneficio empresa COP $ 0,00 | SS+P COP $ 38.000,00 |
ARL COP $ 9.500,00 | ahorro COP $ 19.000,00 | neto COP $ 883.500,00
Caso por comision | bruto COP $ 5.000.000,00 | beneficio empresa COP $ 1.000.000,00 | SS+P COP $
200.000,00 | ARL COP $ 50.000,00 | ahorro COP $ 0,00 | neto COP $ 4.750.000,00
Caso temporal | bruto COP $ 1.800.000,00 | beneficio empresa COP $ 0,00 | SS+P COP $ 72.000,00 |
ARL COP $ 18.000,00 | ahorro COP $ 0,00 | neto COP $ 1.710.000,00
```

Figura 2. Salida de la demostración de nómina para cuatro modalidades de empleo.

8. Desarrollo individual y enlaces

Realicé esta actividad y el desarrollo del sistema de manera individual. Organicé el trabajo en análisis, diseño orientado a objetos, implementación, pruebas y presentación de resultados.

Video de exposición: https://youtu.be/EykNXmOPGHQ

Repositorio del proyecto: https://github.com/Nicand16/sistema-nomina-u3-java

9. Ejecución local

```
mvn clean test
java -cp target/classes co.edu.unicartagena.nomina.cli.PayrollDemo 1.0 5.0
```

Para los parámetros no definidos en el enunciado adopté supuestos de demostración.

Referencias técnicas

Universidad de Cartagena. (s. f.). Actividad Unidad Tres [consigna en PDF proporcionada para esta actividad, 2 páginas].

JUnit Team. (2024). JUnit 5 User Guide, version 5.11.4. https://docs.junit.org/5.11.4/user-guide/index.html